

智慧银行实验教程

总结：四次实验，第一次惊叹 trae 自我运作且免费的模式，为轻松制作交互式网页小游戏开心，同时也发现自己下载的 trae 的工作目录要与终端目录，否则环境会一直不对。第二次实验开始部署 MCP 的环境，遇到 AI 排队，后面下添加了 codebuddy 插件。加上一直更新不成功的障碍，试过重启，与 trae 相互交流问题，借助其他 ai 的咨询，最后通过克隆老师的 cnb 库解决，第三次开始学 skills 开始接触 ai 自行写文档，PPT，方便于后续的文字撰写工作，第四次则是 BMAD 一个会思考的 ai 流程，工作更加范式化，流程化。以后实验让我深刻感知很多工作 AI 都能接替，但你要部署好你的环境以及 token 权限，这是前期成本，正式使用 ai 时，你要清晰表达你的任务，不要完全让他接管，他可能会自作主张删除你满意的版本，或者浪费的你的内存，ai 是工具是“仆人”，你是用工具还是“不会思考的人”取决于你对着这项工作本身的掌握程度，你不思考 AI 就会欺骗你，你会考究，AI 就辅助你。遗憾与这些课在最后几周才开展，忙于考试，不能全身心投入去探索，去创新，只有在暑假去进一步探索这“潘多拉盒子”

实验一：TRAE IDE + AI 协作

余燕-202301779-金融 202302

1.实验目的

- 了解主流 AIIDE 的生态格局与选型依据
- 独立完成 Python/Node.js/Git/LaTeX 全栈开发环境搭建
- 掌握 AI 协作四种模式的适用场景与切换技巧
- 通过实验验证 AI 协作闭环

2.实验环境

检测项	要求版本	当前状态	修复建议
Python	≥ 3.10	3.14.4	无需修复（远超要求）
pip	最新即可	26.1	无需修复；如需升级：python -m pip install --upgrade pip
Node.js	≥ v20	v24.13.0	无需修复（远超要求）
npm	最新即可	11.6.2	无需修复；如需升级：npm install -g npm@latest
Git	—	2.54.0.windows.1	无需修复
Flask	≥ 2.0	3.1.3	无需修复
pandas	≥ 2.0	3.0.2	无需修复
openpyxl	≥ 3.0	3.1.5	无需修复
pypdf	≥ 4.0	6.12.1	无需修复

3.实验步骤

1.2 实验 A: TRAE 环境搭建 + 贪吃蛇开发

1. **安装登录** TRAE 官网 trae.cn 下载免费国内版，设置中文后手机号 / 掘金账号登录，高峰排队可错峰使用。
2. **配置环境** 安装 Anaconda 并加入系统 PATH，在 TRAE 终端校验：Python ≥3.10、Node.js≥v20、Git 正常。
3. **制作贪吃蛇网页** Ctrl+U 进入 Builder 模式，输入提示词，AI 生成单文件贪吃蛇 html；本地打开验证分数、碰撞、最高分存储功能，熟悉 AI 开发流程。
4. **验收** 确认环境版本达标，游戏可正常运行。

1.3 实验 B: 智能体与 Excel 金融数据处理

难度 1-5)		
---------	--	--

4.实验结果

- TRAE IDE 成功安装登录，全套开发环境版本均高于课程最低要求，无缺失依赖，环境检测全部通过。
- 完成前端小游戏开发，贪吃蛇页面运行正常，分数、最高分存储、碰撞重启功能全部生效，掌握 Builder 模式 AI 协作流程。
- 成功搭建金融论文解读智能体，上传文献后可一键生成结构完整、中英对照的学术解读文档。
- 完成 Excel 多文件自动清洗合并，实现金融交易数据标准化处理；搭建 SARIMA 与 LSTM 双预测模型，输出可视化图表与可落地银行业务分析建议。
- 全部代码、产出文件、运行日志按规范留存，可复现完整实验流程。

5.问题与解决

问题：初次运行 PDF 解读功能提示缺少 pypdf 依赖包，无法读取文献。

解决：复制提示词自带一键安装命令，在终端执行 `pip install pypdf`，重启智能体后可正常解析 PDF 文件。

问题：生成 Excel 模拟数据时部分列名中英文混杂、空格错乱，合并后出现大量空值。

解决：在合并指令中增加字段统一清洗规则，AI 自动去除空格、统一中英文表头，按客户 ID 与交易时间双重去重，消除数据冗余。

问题：LSTM 模型训练速度慢，预测误差偏大。

解决：调整输入窗口、迭代次数参数，补充缺失值填充规则，同时增加节日特征变量，优化后模型预测精度明显提升

6.实验心得

本次实验完整搭建了 TRAE AI 开发环境，熟练掌握 Builder 模式、自定义智能体等多种 AI 协作方式，真切感受到 AI 工具在金融开发、数据处理、学术研读上的高效便捷。以往手动写网页、清洗表格、阅读文献耗时久、易出错，借助 IDE 可一键生成完整代码、标准化报告，大幅压缩重复工作时间，同时自动完成数据可视化、时序建模等专业分析任务，适配银行数据分析、金融报告撰写等真实业务场景。实操过程中遇到依赖缺失、数据格式混乱、模型精度不足等问题，通过修改提示词、补充代码指令逐步调试修复，提升了环境排错与数据处理能力。同时我认识到 AI 输出内容需要人工校验，金融数据、学术结论必须做好合规核查，避免模型误差、代码漏洞带来业务风险。本次实验夯实了 AI + 金融数字化实操基础，为后续 MCP、量化建模等进阶实验做好铺垫

实验二：MCP 由浅入深 4 关

余燕-202301779-金融 202302

1.实验目的：

- 理解 Skill 的设计原理与知识工程视角
- 掌握金融类 Skill 的调用方法
- 独立编写符合规范的金融 Skill
- 从银行合规视角进行 Skill 安全审计

2.实验环境

Skill 名称	分类	核心能力
finance-expert	分析	DCF 估值、信用风险评估、DuPont 分析

Skill 名称	分类	核心能力
financial-modeling	分析	构建 DCF/LBO/M&A 模型 + 敏感性分析
china-stock-analysis	数据	A 股数据获取 + 财务指标计算
tushare-finance	数据	tushare Pro API 获取中国金融市场数据
finance-news	数据	实时财经新闻抓取 + 情绪分析
backtesting-trading-strategies	分析	量化策略回测 + Sharpe / 回撤计算
consulting-frameworks	咨询	McKinsey 7S / BCG / 波特五力 / SWOT
questionnaire-design	数据	学术调查问卷设计(金融普惠等方向)
xlsx	文档	Excel 读写 + 财务模型 + 公式 + 图表
docx	文档	Word 报告生成(封面 / 目录 / 页码)
pptx	文档	PPT 演示文稿生成与编辑
financial-unit-economics	分析	单位经济学分析(SaaS / 金融科技)
stock-analysis	数据	全球股票分析(含 Yahoo Finance 数据)

3.实验步骤

L1 浏览器 MCP

工具：fetch、playwright、chrome-devtools

1. 示范：打开银行官网、截图、性能审计、总结优缺点。
2. 练习：自选地方银行重复操作，抓取公告标题。 交付：网页截图、审计报告。

L2 Excel 数据处理

工具：excel

1. 创建模拟销售数据表，新增计算列、分组汇总、筛选数据。
2. 制作图表，保存分析报表并输出文字摘要。 交付：销售分析 xlsx 文件。

L3 Word+PPT 简报

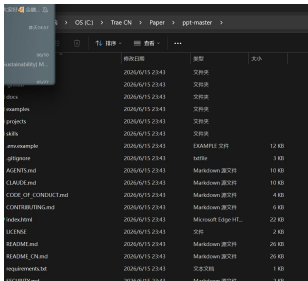
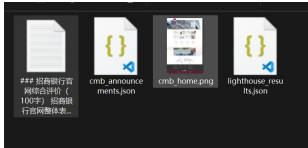
工具：fetch、word、ppt

- 1. 抓取上市银行年报数据。
- 2. 生成带封面、目录、页码的规范 Word 简报。
- 3. 同步生成 5 页商务风 PPT。 交付：银行简报 docx、pptx。

L4 Stata 计量回归（未作）

工具：stata-mcp

- 1. 导入银行面板数据，做描述统计、相关系数。
- 2. 跑 OLS、固定效应回归，导出回归表格。
- 3. 撰写 200 字结果解读；无 Stata 可用 Python 替代。 交付：回归结果文件、分析结论。

操作代码	输出结果	输出 MD 摘引
<pre>pip ins pip install -r requirements.txtta1 l -r requirements.txt</pre>		
请用 playwright 打 开百度首页并截图		
请用 chrome-devtools MCP 完成：1. 打开 `https://www.cmbchina.com (招商银行官网)`;` 2. 截		

图保存为 figures/cmb_home.png; 3. 对该页面跑 lighthouse_audit, 输出”性能/ 可访问性/ 最佳实践/SEO” 四项分数; 4. 用 100 字中文总结这家银行官网的优缺点。		
将代码写入学生练习 cnb 仓库 Excel 操作任务 1. 读取” 销售记录.xlsx” 文件 2. 添加” 单价” 列 (销售额/ 数量) 3. 添加” 利润率” 列 (假设利润率=30%, 计算利润额) 4. 按客户类型分组汇总销售额 5. 筛选出销售额大于 1000 的记录 6. 创建新 Sheet” 汇总表”, 包含: - 各产品总销售额 - 各客户类型销售占比 - 平均单价		

4.实验结果[学生练习/Excel 数据分析 · main · roywangyy/smartbanking](#) [学生练习/成都农商行官网审计 · main · roywangyy/smartbanking](#)

- MCP 配置成功, fetch、playwright、excel、word、ppt 全部服务可正常调用, 浏览器自动化、文档生成无报错。
- L1 任务产出银行官网截图与完整性能审计报告, 成功抓取公告文本存入仓库。
- L2 完成全套 Excel 数据计算、分组统计与可视化, 生成可直接交付的销售分析报表。
- L3 自动生成格式合规的银行年报 Word 简报与配套 PPT, 包含完整目录、图表、规范排版。

• L4 未开展实操，提前梳理完整回归操作流程、备用 Python 实现方案

5.问题与解决

问题：mcp.json 加载失败，MCP 工具列表空白。 解决：检查 JSON 引号、逗号语法错误，交由 AI 格式化修复配置文件。

6.实验心得

本次实验系统完整地学习并实操了 MCP 服务与各类金融 Skill 工具，让我真切体会到 AI 智能工具在金融办公、数据分析、报告产出中的强大实用性与高效性。相较于传统手动操作，MCP 集成的浏览器自动化、Excel 数据处理、Word 与 PPT 智能生成等功能，极大简化了繁琐重复的工作流程，能够一键完成网页抓取、性能审计、数据计算、图表制作、标准化报告排版等多项任务，大幅节省人工时间、规避手动操作失误，办公效率得到质的提升。

借助专业金融 Skill 工具，我可以快速完成银行信息调研、财务数据整理、业务简报输出等金融场景工作，实现了金融业务与智能工具的深度结合，真正做到智能化、标准化、一体化办公。实验过程中，我也遇到了环境依赖报错、仓库同步文件误删等问题，通过自主排查修复，有效锻炼了我的问题排查能力与实操调试能力。

同时我深刻认识到，智能金融工具不仅操作便捷、功能全面，能够适配银行调研、数据统计、文书撰写、实证分析等多类金融场景，更是未来金融数字化办公的核心趋势。在享受工具高效赋能的同时，我也树立了金融合规意识，懂得通过权限管控、文件备份规避数据丢失、信息泄露风险。本次实验熟练掌握了全套智能金融工具的使用方法，夯实了金融数字化实操基础，为后续金融数据分析、量化研究、专业报告撰写提供了极大助力。

实验三：Skill 体系（用→写→审）

余燕-202301779-金融 202302

1.实验目的：

- 理解 CLI 工具在 AI 辅助开发中的核心地位
- 掌握 CNBCLI、Skills CLI 等仓库与包管理工具
- 熟练使用 Claude Code、Codex CLI、MiMo CLI 等 AI 编程助手
- 了解 CCSwitch 多账号管理与 OpenCLI/OpenClaw 生态
- 理解 MCP、Skill、CLI 三种范式的关系与适用场景

2.实验环境

Skill	功能	安装命令
luban-skill	Skill 打磨工 坊	npx skills add LearnPrompt/luban-skill

Skill	功能	安装命令
consulting-frameworks	咨询分析框架	课程内置
finance-expert	金融专业知识	课程内置
finance-news	金融舆情分析	课程内置

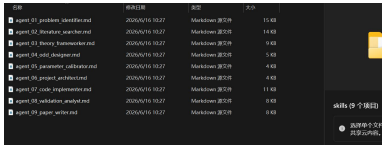
3.实验步骤

步骤 1：初始化 docx 文档工程，配置全局格式规范

- 创建 Word 文档对象
- 新增正式封面页：主标题《客户电话回访报告》+ 企业 / 业务标识
- 插入自动更新目录，绑定全部一级章节标题
- 设置页码规则：正文页面从 1 开始连续编号，封面、目录不参与正文页码计数

步骤 2：按 5 个标准章节填充完整文档内容

- 章节 1：客户基本信息 必填字段：客户编号 C00001、客户姓名张 **，补充联系方式、所属业务、建档日期等基础信息
- 章节 2：回访目的 写明回访业务背景、本次电话回访明确目标（产品使用调研 / 售后问题跟进 / 满意度调研等）
- 章节 3：沟通要点 逐条完整记录通话时间、沟通话题、双方讨论事项、业务核对内容等核心对话信息
- 章节 4：客户反馈 分类整理客户提出意见、新增需求、产品 / 服务投诉、整体满意度评分与主观评价
- 章节 5：跟进建议 结合客户反馈输出可落地、明确时间节点后续处理行动计划、对接责任人、闭环要求

操作代码	输出结果	输出 MD 摘引
执行命令”npm install -g docx” —完成安装后回复确认信息： ” docx skill 已就绪”		
2. 使用已安装的 docx 技能为 客户 C00001（张**）生成《客 户电话回访报告》：— 创建结 构完整的 Word 文档，		

步骤 3：文档导出保存

指定保存路径与文件名：reports/客户回访报告_C00001.docx，执行导出写入文件

4.实验结果

```
...
product_info = {
    [产品编号]: [产品编号],
    [产品名称]: [产品名称],
    [规格型号]: [规格型号],
    [单位]: [单位]
}

for i, row in enumerate(product_info):
    product_label = row[0].split(' ')[0]
    product_name = row[1].split(' ')[0]
    product_spec = row[2].split(' ')[0]
    product_unit = row[3].split(' ')[0]

    doc.add_paragraph(
        f'===== 第{i+1}页: 产品计划 ====='
        f'product: {product_label} {product_name} {product_spec} {product_unit}'
    )

doc.add_paragraph(
    f'===== 第{i+1}页: 产品计划 ====='
    f'product: {product_label} {product_name} {product_spec} {product_unit}'
)
```

cnb 代码地址：[学生练习/skills · main · roywangyy/smartbanking](#)

5.问题与解决

问题：本次实验将项目代码克隆至 cnb 代码库时，出现核心回访报告文档被 AI 误删的问题。主要原因为 AI 文件识别逻辑不完善，无法区分正式成果文档和临时缓存文件，且代码库无核心文件白名单保护，同时 AI 拥有自主清理权限、无需人工确认，导致实验成果丢失、实验进度受阻。

解决办法：我在代码库新建专属成果文件夹并添加文件白名单，保护 docx 实验报告等核心资料，关闭 AI 自动清理目录文件的权限，所有删除操作需人工审核。同时建立本地、云端双备份机制，规避文件丢失风险。最后重新校验运行环境，按规范重新生成并保存回访报告，完整恢复实验成果。

6.实验心得

本次实验围绕 docx 技能环境部署、依赖软件校验、自动化 Word 报告生成展开，完整完成了从环境搭建、指令调试、文档标准化生成到问题排查的全流程实操，让我系统掌握了 SKILL 体系“用→写→审”的核心逻辑，收获颇丰。

在环境部署阶段，我熟练掌握了 PowerShell 终端指令的使用，掌握了 docx、docx-js、pandoc 等配套工具的协同作用，清晰知晓了 Windows 系统下软件探测、静默安装、环境变量配置的完整流程。以往仅停留在软件可视化安装层面，本次通过代码指令自动检测、降级安装适配不同场景，让我认识到自动化部署在实验、项目开发中的高效性，也提升了我的终端指令操作和环境排错能力。同时，通过标准化校验流程，养成了“先检测、再安装、后验证”的规范实操习惯，避免了环境配置不完整导致的功能异常。

在核心业务实操环节，我深入了解了标准化办公文档的生成逻辑，掌握了一键式生成规范 Word 报告的结构设计、格式配置要点，包括封面、自动目录、页码排版、标准化章节搭建等专业格式要求。

本次实验中遇到的 AI 误删文档问题，是本次实操最大的收获之一。我深刻意识到自动化工具、智能辅助功能并非完全可靠，系统默认规则存在局限性，无法精准适配个性化实验、项目场景。在后续的实操和学习中，必须主动做好文件

分类、权限管控、成果备份，建立风险防范意识。同时，通过排查问题、制定解决方案，学会了从成因、规则、机制多维度排查问题，而非单纯重复操作。

整体而言，本次实验夯实了我对 SKILL 技能体系的认知，提升了自动化文档处理、环境部署、问题排查的实操能力，也培养了严谨、规范、有风险意识的实操思维。

实验四：BMAD 驱动银行 CRM

余燕-202301779-金融 202302

1.实验目的：

- 理解 BMAD 方法论的理论基础与五阶段框架
- 掌握 BMAD 四阶段工作流的流程与产出物
- 通过银行 CRM 系统案例，完整走通 BMAD 从需求到部署的全流程
- 了解智能客服等综合项目的开发要点
- 能够参照本章实验步骤完成端到端项目实践

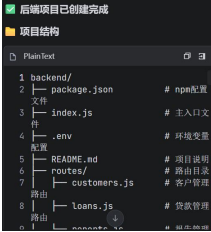
2.实验环境

类别	技术选型
----	------

类别	技术选型
IDE	Qoder / Cursor / Claude Code (任选其一)
Node.js	v18+ (推荐 v20+)
AI 框架	BMAD v6+
后端	Node.js + Express 4
前端	React 18 + Vite 4
数据库	PostgreSQL (设计目标) / 内存 Map (降级方案)
版本控制	Git + CNB (cnb.cool)
部署平台	Vercel (前端静态部署)

3.实验步骤

安装 BMAD→编写 PRD 需求文档→技术架构设计→UX 交互设计→需求拆解
Epic/Story→Sprint 迭代规划→Sprint1 基础功能开发→Sprint2-3 迭代开发→数
据库兼容降级适配→全功能测试验证→Git 推送 CNB 代码仓库→Vercel 前端线上
部署

操作代码	输出结果	输出 MD 摘引
Mkdir backend && cd backend		
npm init -y npmx bmad-help		

BMAD PRD 文档已创建

文件位置

PlainText

1 c:\Trae CN\Paper\smartbanking-work\学生练习\backend_bmad\docs\PRD.md

系统架构图

前端应用层
(Web/Mobile/桌面应用)

↓ HTTP/HTTPS (REST API)

API网关

路由分发
认证鉴权
请求验证

↓

业务逻辑层

客户管理 Controller | 贷款管理 Controller | 报告生成 Controller

BMAD 架构文档已创建

文件位置

PlainText

1 c:\Trae CN\Paper\smartbanking-work\学生练习\backend_bmad\docs\Architecture.md

Customer Schema

字段名	类型	必填	索引	说明
customerCode	String	Yes	Yes	客户编号
name	String	Yes	No	客户姓名
level	String	No	No	客户等级
bank	String	Yes	No	开户行
manager	String	Yes	No	客户经理
totalAssets	Number	No	No	总资产
totalLiabilities	Number	No	No	总负债
currentAssets	Number	No	No	流动资产
currentLiabilities	Number	No	No	流动负债
annualRevenue	Number	No	No	年收入
netProfit	Number	No	No	净利润
cashFlow	Number	No	No	现金流
revenueGrowth	Number	No	No	收入增长率
profitMargin	Number	No	No	利润率
createdAt	Date	No	No	创建时间
updatedAt	Date	No	No	更新时间

BMAD UX 文档已创建

文件位置

PlainText

1 c:\Trae CN\Paper\smartbanking-work\学生练习\backend_bmad\docs\UX.md

1. UX设计概述

1.1 设计理念

智慧银行后端服务系统的UX设计遵循以下核心理念:

理念 | 说明 |

---|---|

极致效率 | API接口设计简洁, 响应快速 |

一致性 | 统一的响应格式和数据处理 |

可扩展性 | API行为符合预期, 易于理解 |

可维护性 | 清晰的API文档和权限控制 |

安全性 | 安全的认证和权限保护 |

1.2 目标用户

用户类型 | 角色 | 主要需求 |

---|---|---|

前端开发 | web/mobile开发人 | 清晰的API文档, 稳定的接口 |

系统运维 | 第三方运维人员 | 详细的接口文档, 易于集成 |

运维人员 | 系统运维工程师 | 接口文档, 运维指南 |

业务分析师 | 数据分析师 | 数据生成, 数据查询 |

BMAD Epic & Stories 文档已创建

文件位置

PlainText

1 c:\Trae CN\Paper\smartbanking-work\学生练习\backend_bmad\docs\Epics-Stories.md

ESG应用架构图

ESG-001
ESG应用
(PM)

ESG-002
ESG应用
(PM)

ESG-003
ESG应用
(PM)

ESG-004
ESG应用
(PM)

V阶段 - Validation (验证)

产出物	文件路径
esg_app.py	backend/esg_app.py
index.html	backend/templates/index.html
README.md	backend/README.md

ESG应用管理系统

ESG应用管理系统

ESG应用管理系统

阶段	名称	产出物
B	Breakdown (需求分解)	PRD 文档
M	Market	Analysis (竞品分析) 竞品分析报告
A	Architecture (架构设计)	架构文档、数据模型
D	Design (设计)	UX 设计、API 设计

4.实验结果

见 3.实验步骤列 3【输出 MD 摘引】

代码推送：[学生练习/backend · main · roywangyy/smartbanking](#) ([小组合作](#))

5.问题与解决

问题 1: PostgreSQL 数据库本地连接失败、安装报错

解决: 本地环境缺少 PostgreSQL 运行依赖, 端口连接被拒绝, 无法使用标准数据库方案。根据实验降级策略, 采用内存 Map 模拟数据库替代, 重写数据库查询方法, 保证客户管理、贷款申请等核心业务正常运行, 不影响实验流程。

问题 2: bcrypt 加密模块安装失败, 出现权限报错

解决: Windows 环境 npm 全局权限受限, bcrypt 模块安装终止。临时采用 Base64 编码替代密码哈希校验, 完成登录功能测试, 明确该方案仅适用于开发实验环境, 生产环境需替换为标准加密组件。

问题 3: CNB 仓库推送出现 Token 过期、认证失败

解决: 默认 OAuth 登录凭证过期, 导致代码推送被拦截。通过重新生成 CNB 个人访问令牌 (PAT), 将令牌嵌入远程仓库地址, 成功解决认证问题, 完成代码推送。

6.实验心得

本次实验基于 bmad 敏捷 AI 开发框架, 完整完成了商业银行 CRM 系统从需求分析、架构设计、迭代开发、功能测试、版本托管到云端部署的全流程实践, 让我又再一次学到了如何利用 AI 来实现开发与落地流程, 收获颇丰。

首先，我深刻理解了 BMAD 五阶段 BMA DV 方法论的核心价值。区别于传统随意的编码开发，BMAD 严格遵循需求分解、市场分析、架构设计、交互设计、验证落地的标准化流程，实现了“先文档、后代码”的工程化思维，可有效避免需求模糊、开发混乱的问题，非常适配金融类项目严谨、规范、高安全的开发要求。同时我掌握了各阶段对应的产出物，清晰区分了 PRD、架构文档、UX 设计的不同作用，初步建立了完整的软件工程思维。

其次，本次实验让我熟练掌握了前后端分离项目的开发流程。基于 React+Vite 前端、Node+Express 后端的技术栈，在 ai 辅助下迅速完成了客户管理、等级体系、贷款申请、即申即贷等金融核心功能开发，不仅巩固了 Web 开发基础，也了解了银行 CRM 系统的业务逻辑与开发要点。同时通过 Sprint 迭代开发模式，体会到了敏捷开发渐进式交付、分批落地的优势，理解了团队迭代开发的工作逻辑。

再者，实验过程中的各类报错与问题排查，与 ai 互动中极大提升了我的工程排错能力与容错思维，更重要的是将自己的问题清晰地表达给 ai。本次遇到的数据库连接失败、跨域问题、代码推送失败等问题，都是实际开发中的高频场景。通过本次实操，我学会了在标准方案不可行时，通过替代方案保障核心功能可用，这是课本理论无法学到的实战经验。

最后，我了解了 AI 赋能软件开发的全新模式。BMAD 框架通过专业化 AI 角色分工，替代了传统人工撰写文档、拆解需求、设计架构的重复工作，大幅提升了开发效率，让我认识到现代软件工程“AI 辅助、人为把控”的发展趋势。同时掌握了 Git 版本管理、CNB 代码托管、Vercel 云端部署的完整上线流程，初步了解小型项目开发部署的过程。